

11. Un compteur qui déborde à 255

Ce n'est pas un bogue : c'est ce que fait un octet.

1. Ce qu'il faut comprendre

`uint8_t` est **un octet** : de 0 à 255. À 255, `compteur++` ramène à 0 — et tout le projet est bâti là-dessus.

L'ordre compte. `image()` est **à la fin** : on dessine, puis on attend. Mettre l'attente en premier ferait sauter la première image.

Laisser tourner quelques secondes suffit à voir le compteur repasser par zéro.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1289 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t compteur = 0;

    while (true) {
        nombre(8, 8, compteur);
        compteur++;
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



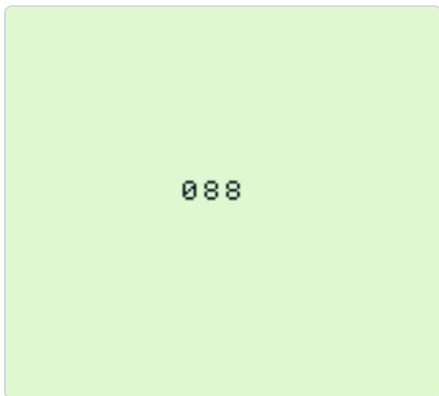
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un nombre qui monte, et qui repart de 000 après 255.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le compteur monte (9 → 29)
- ✓ il reste dans un octet
- ✓ et il est affiché

Ensuite : **12. Un dé, et le hasard du matériel** — niveau 2 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++